

Resumen

Los mercados de activos digitales mueven cientos de millones anuales y aún así carecen de herramientas analíticas y de automatizaciones integradas o de libre uso. La motivación principal de este proyecto nace de esta profunda brecha tecnológica. Actualmente, la gestión de carteras en la mayoría de los mercados se realiza de forma manual o mediante herramientas estáticas, lo que resulta altamente ineficiente y riesgoso.

El propósito principal de este TFM es evaluar si un ecosistema descentralizado de agentes de inteligencia artificial puede superar las limitaciones humanas y generar rendimientos estables en mercados de alta fricción. Se opta por una arquitectura multiagente de Aprendizaje por Refuerzo para descentralizar la toma de decisiones y reducir la complejidad computacional del mercado. El objetivo es que los agentes aprendan estrategias de inversión óptimas que maximicen el saldo total de la cartera.

La metodología a desarrollar en el TFM se aplicará al ámbito de la economía virtual de Counter-Strike 2, un popular videojuego que cuenta con un ecosistema financiero integrado de alta frecuencia y liquidez.

Palabras clave: Mercados de activos digitales; de decisiones; Arquitectura multiagente; aprendizaje por refuerzo.

Resum

Els mercats d'actius digitals mouen centenars de milions anuals i, tot i això, encara no disposen d'eines analítiques ni d'automatitzacions integrades o de lliure ús. La motivació principal d'aquest projecte naix d'aquesta profunda bretxa tecnològica. Actualment, la gestió de carteres en la majoria dels mercats es realitza de forma manual o mitjançant eines estàtiques, la qual cosa resulta altament ineficient i arriscada.

El propòsit principal d'aquest TFM és avaluar si un ecosistema descentralitzat d'agents d'intel·ligència artificial pot superar les limitacions humanes i generar rendiments estables en mercats d'alta fricció. S'opta per una arquitectura multiagent d'aprenentatge per reforç per descentralitzar la presa de decisions i reduir la complexitat computacional del mercat. L'objectiu és que els agents aprenguen estratègies d'inversió òptimes que maximitzen el saldo total de la cartera.

La metodologia a desenvolupar en el TFM s'aplicarà a l'àmbit de l'economia virtual de Counter-Strike 2, un videojoc popular que compta amb un ecosistema financer integrat d'alta freqüència i liquiditat.

Paraules clau: Mercats d'actius digitals; presa de decisions; Arquitectura multiagent; aprenentatge per reforç.

Abstract

Digital asset markets move hundreds of millions annually and yet still lack integrated or freely available analytical tools and automation systems. The main motivation of this project stems from this deep technological gap. At present, portfolio management in most markets is carried out manually or through static tools, which makes it highly inefficient and risky.

The main purpose of this master's thesis is to evaluate whether a decentralized ecosystem of artificial intelligence agents can overcome human limitations and generate stable returns in high-friction markets. A multi-agent reinforcement learning architecture is chosen to decentralize decision-making and reduce the computational complexity of the market. The goal is for the agents to learn optimal investment strategies that maximize the total portfolio balance.

The methodology to be developed in this master's thesis will be applied to the virtual economy of Counter-Strike 2, a popular video game with an integrated financial ecosystem characterized by high frequency and liquidity.

Key words: Digital asset markets; decision-making; Multi-agent architecture; reinforcement learning.

Índice general

Índice de figuras	6
Índice de tablas	7
Lista de algoritmos	8
Lista de acrónimos	9
1 Introducción	10
1.1 Motivación	10
1.2 Planteamiento del problema	11
1.3 Objetivos	11
1.4 Alcance y restricciones	11
1.5 Metodología de desarrollo	12
1.6 Estructura del documento	12
2 Marco teórico y estado del arte	13
2.1 Mercados de activos digitales	13
2.1.1 Activos virtuales en Counter-Strike 2	13
2.1.2 Fricciones de mercado	13
2.2 Aprendizaje por refuerzo	13
2.2.1 Procesos de decisión de Markov	13
2.2.2 Política, retorno y exploración	13
2.3 Aprendizaje por refuerzo multiagente	13
2.3.1 Cooperación y coordinación entre agentes	13
2.3.2 Entrenamiento centralizado y ejecución descentralizada	13
2.4 Toma de decisiones en carteras	13
2.4.1 Rentabilidad y riesgo	13
2.4.2 Liquidez y costes de transacción	13
2.5 Aplicaciones relacionadas	13
3 Arquitectura multiagente propuesta	14
3.1 Definición del problema	14
3.2 Visión general del sistema	14
3.3 Definición del entorno	14
3.3.1 Estado y observaciones	15
3.3.2 Espacio de acciones	15
3.3.3 Transición del entorno	15
3.4 Agentes del sistema	15
3.4.1 Scout	16
3.4.2 Trader	16
3.4.3 Portfolio	16
3.5 Función de recompensa	16
3.5.1 Recompensa individual	16
3.5.2 Recompensa cooperativa	16
3.6 Restricciones de riesgo	16

4	Configuración del entorno y datos	18
4.1	Fuentes de datos	18
4.1.1	Mercado de Steam	18
4.1.2	Mercados externos	18
4.2	Adquisición y persistencia	18
4.2.1	Pipeline de adquisición	18
4.2.2	Modelo de persistencia	19
4.3	Preparación de características	19
4.3.1	Variables de mercado	19
4.3.2	Variables de cartera	19
4.4	Simulador de cartera	19
4.4.1	Operaciones permitidas	19
4.4.2	Comisiones y valoración	20
4.5	Interfaz de consulta	20
5	Experimentación y análisis	21
5.1	Diseño experimental	21
5.1.1	Escenarios de evaluación	21
5.1.2	Estrategias de comparación	21
5.2	Métricas	21
5.2.1	Métricas de rentabilidad	21
5.2.2	Métricas de riesgo	22
5.3	Resultados	22
5.3.1	Resultados del modelo supervisado	22
5.3.2	Dataset de trading real	22
5.3.3	Resultados de la arquitectura multiagente	22
5.4	Discusión	23
5.4.1	Casos favorables	23
5.4.2	Limitaciones observadas	23
6	Conclusiones	24
6.1	Conclusiones principales	24
6.2	Cumplimiento de objetivos	24
6.3	Limitaciones	24
6.4	Trabajo futuro	25
	Bibliografía	26
A	Anexos	27
A.1	Estructura del repositorio	27
A.2	Configuración del entorno	27
A.3	Parámetros de entrenamiento	28
A.4	Modelo de datos	28
A.5	Pruebas implementadas	28
A.6	Resultados adicionales	28

Índice de figuras

3.1	Flujo general de la arquitectura propuesta.	14
-----	---	----

Índice de tablas

5.1	Resultados supervisados preliminares sobre el dataset reciente.	22
-----	---	----

Lista de algoritmos

1	Ciclo general de toma de decisiones	15
---	---	----

Lista de acrónimos

API Interfaz de programación de aplicaciones.

CS2 *Counter-Strike 2*.

IA Inteligencia artificial.

MARL Aprendizaje por refuerzo multiagente, del ingles *Multi-Agent Reinforcement Learning*.

MDP Proceso de decisión de Markov, del ingles *Markov Decision Process*.

PPO *Proximal Policy Optimization*.

RL Aprendizaje por refuerzo, del ingles *Reinforcement Learning*.

Introducción

1.1 Motivación

Los mercados de activos digitales asociados a videojuegos han dejado de ser un fenómeno marginal. En el caso de *Counter-Strike 2*, los objetos cosméticos como skins, cuchillos, guantes, pegatinas o cajas se compran y venden de forma continua en diferentes plataformas. Aunque estos activos no tienen una forma física, su precio se comporta como una señal económica observable: cambia con la demanda, la rareza, la liquidez, la condición del objeto, la disponibilidad en cada mercado y las expectativas de reventa.

El interés de este TFM nace de una contradicción bastante clara. Por un lado, existe un mercado activo, con histórico de precios, diferencias entre plataformas y oportunidades de arbitraje. Por otro, buena parte de la gestión real sigue dependiendo de hojas de cálculo, comprobaciones manuales, enlaces guardados y reglas estáticas. Esta forma de trabajo puede servir para un número pequeño de artículos, pero se vuelve frágil cuando se intenta seguir muchos objetos a la vez, comparar monedas, estimar comisiones, controlar capital bloqueado y reaccionar ante cambios de precio.

El caso de *Counter-Strike 2* añade además varias fricciones que hacen que una oportunidad aparente no siempre sea una buena decisión. Comprar un artículo barato no basta: hay que considerar la comisión de la plataforma, la conversión entre EUR y CNY, el margen real de salida, el volumen disponible, el *trade hold*, el tiempo probable hasta la venta y el coste de oportunidad del capital inmovilizado. En la práctica, una operación rentable en bruto puede dejar de serlo cuando se calcula el beneficio neto.

Antes de este proyecto, parte de estas decisiones ya se apoyaban en un Excel operativo con fórmulas de conversión de moneda, comisiones, beneficio, porcentaje de retorno, fechas de desbloqueo y capital en espera. Ese Excel ha servido como especificación inicial del dominio. No se ha copiado como estructura final, porque mezclaba registro operativo, calculadoras y vistas manuales, pero sí ha permitido identificar las reglas económicas que debían convertirse en funciones reutilizables dentro del sistema.

La motivación técnica del trabajo es transformar ese proceso manual en una plataforma reproducible de adquisición, simulación y decisión. El objetivo no es ejecutar compras reales durante el desarrollo del TFM, sino construir un entorno controlado en el que se puedan evaluar estrategias con datos históricos y reglas de mercado realistas. Sobre esa base se plantea una arquitectura multiagente en la que distintos componentes se especializan en observar el mercado, producir señales, simular operaciones y controlar el riesgo de cartera.

1.2 Planteamiento del problema

El problema que aborda este TFM puede resumirse como una tarea de decisión secuencial con información incompleta. En cada momento existe un conjunto de activos candidatos, varias fuentes de precio, señales de liquidez y un estado de cartera que condiciona la siguiente acción. La decisión no consiste únicamente en clasificar si un precio subirá, sino en decidir si una operación tiene sentido después de aplicar comisiones, cambio de divisa, bloqueo temporal y restricciones de exposición.

Esta formulación obliga a separar dos niveles. El primero es el nivel de observación y predicción: recopilar datos, limpiarlos, construir características y estimar probabilidades útiles. El segundo es el nivel de decisión: usar esas señales dentro de un simulador de cartera que permita comprar, vender, mantener u observar sin arriesgar capital real. La arquitectura multiagente se introduce en este segundo nivel como hipótesis de trabajo para repartir responsabilidades y reducir la complejidad de una decisión centralizada.

1.3 Objetivos

El objetivo principal de este TFM es diseñar y evaluar una arquitectura multiagente para apoyar la toma de decisiones en mercados de activos digitales, usando como caso de estudio la economía virtual de *Counter-Strike 2*.

Los objetivos específicos se agrupan en cuatro bloques:

- **Comprensión del dominio:** analizar el mercado de activos digitales de *Counter-Strike 2*, sus plataformas, restricciones, comisiones, divisas, liquidez y efectos del *trade hold*.
- **Construcción de datos:** diseñar un flujo de adquisición y preparación de datos procedentes de Steam, BUFF163, SteamDT, OCR y fuentes manuales, manteniendo trazabilidad de fuente, fecha, moneda y calidad del dato.
- **Simulación y predicción:** migrar reglas económicas del proceso manual a código comprobable, construir datasets temporales, entrenar modelos supervisados y generar señales probabilísticas que puedan usarse dentro del entorno de decisión.
- **Decisión multiagente:** formular el problema como un entorno de aprendizaje por refuerzo multiagente, definir agentes especializados, establecer recompensas coherentes con la cartera y evaluar el comportamiento mediante métricas de rentabilidad, riesgo, exposición y capital inmovilizado.

1.4 Alcance y restricciones

El alcance del proyecto se centra en la simulación y en la evaluación. Durante el desarrollo del TFM no se ejecutan compras reales ni se mueve capital real. Cualquier decisión generada por el sistema se registra como recomendación o decisión simulada. Esta restricción es importante porque permite experimentar con estrategias sin introducir riesgos económicos ni depender de una integración operativa completa con plataformas externas.

El proyecto tampoco asume que el aprendizaje por refuerzo multiagente sea útil por definición. La arquitectura se plantea como hipótesis de trabajo: si los datos, el simulador y las recompensas no son suficientemente informativos, el sistema debe mostrarlo en

los resultados. Por este motivo se mantiene una separación explícita entre adquisición de datos, predicción supervisada, simulador económico, agentes MARL y evaluación experimental.

Otra restricción relevante es la disponibilidad de histórico alineado entre plataformas. El mercado de Steam y el de BUFF163 no siempre ofrecen datos con la misma granularidad, moneda, horizonte o estabilidad de acceso. Por ello, una parte del trabajo consiste en construir una base experimental honesta, donde los resultados preliminares se distingan claramente de una validación definitiva.

1.5 Metodología de desarrollo

La memoria se organiza siguiendo el propio avance del proyecto. Primero se estudió el proceso manual y se extrajeron las reglas económicas del Excel operativo. Después se construyó una base de adquisición y persistencia para recoger datos de mercado. A continuación se generaron datasets supervisados y se evaluaron modelos tabulares para obtener señales probabilísticas. Finalmente se implementó un simulador de cartera y un entorno multiagente compatible con PettingZoo y RLlib para realizar pruebas funcionales.

Esta metodología incremental evita entrenar agentes sobre un entorno poco fiable. En lugar de empezar directamente por MARL, el proyecto construye primero las piezas que hacen verificable la decisión: datos, reglas económicas, simulación, métricas y trazabilidad. La arquitectura multiagente se apoya en estas capas, no las sustituye.

1.6 Estructura del documento

El Capítulo 2 presenta el marco teórico necesario para entender el proyecto: mercados de activos digitales, aprendizaje por refuerzo, aprendizaje por refuerzo multiagente, toma de decisiones en carteras y trabajos relacionados.

El Capítulo 3 describe la arquitectura multiagente propuesta. En él se define el problema, el flujo general del sistema, el entorno de decisión, los agentes, la función de recompensa y las restricciones de riesgo.

El Capítulo 4 se centra en los datos y en el entorno experimental. Se explican las fuentes de adquisición, la persistencia, las características utilizadas, el simulador de cartera, el OCR y la interfaz de consulta.

El Capítulo 5 recoge la experimentación realizada hasta el momento. Incluye la migración de reglas económicas, los datasets supervisados, los modelos tabulares, el dataset de trading real, el estado del entrenamiento MARL y las limitaciones detectadas.

Por último, el Capítulo 6 resume las conclusiones parciales, revisa el grado de cumplimiento de los objetivos y plantea las líneas de trabajo necesarias para cerrar la validación experimental.

Marco teórico y estado del arte

2.1 Mercados de activos digitales

2.1.1 Activos virtuales en Counter-Strike 2

2.1.2 Fricciones de mercado

2.2 Aprendizaje por refuerzo

2.2.1 Procesos de decisión de Markov

2.2.2 Política, retorno y exploración

2.3 Aprendizaje por refuerzo multiagente

2.3.1 Cooperación y coordinación entre agentes

2.3.2 Entrenamiento centralizado y ejecución descentralizada

2.4 Toma de decisiones en carteras

2.4.1 Rentabilidad y riesgo

2.4.2 Liquidez y costes de transacción

2.5 Aplicaciones relacionadas

Arquitectura multiagente propuesta

3.1 Definición del problema

El problema se formula como una tarea de decisión secuencial sobre una cartera simulada de activos digitales. En cada instante, el sistema recibe observaciones de mercado, estado de cartera y señales derivadas. A partir de esa información debe decidir si conviene comprar, vender, mantener en observación o rechazar una oportunidad.

La dificultad no está únicamente en predecir si un precio subirá. Una decisión operativa debe considerar el beneficio neto después de comisiones, el cambio de moneda, la liquidez disponible, el capital ya bloqueado, el tiempo de desbloqueo por *trade hold* y la exposición acumulada a un mismo activo o plataforma. Por ello, el objetivo del sistema es maximizar el valor total de la cartera simulada bajo restricciones de riesgo y fricción de mercado.

3.2 Visión general del sistema

La arquitectura separa cinco capas principales: adquisición, persistencia, predicción, decisión multiagente y simulación. Esta separación evita que los agentes de aprendizaje mezclen responsabilidades que no les corresponden. Los extractores observan fuentes externas; el modelo supervisado calcula señales probabilísticas; el entorno MARL convierte esas señales en observaciones; y el simulador aplica las consecuencias económicas de cada acción.

Fuentes externas → extractores Steam/BUFF/OCR/SteamDT → histórico de mercado → modelo supervisado calibrado → entorno PettingZoo → Scout, Trader y Portfolio → decisión simulada → simulador de cartera

Figura 3.1: Flujo general de la arquitectura propuesta.

La decisión final siempre queda registrada como simulada. Esto permite evaluar el sistema con métricas de cartera sin exponer capital real durante la investigación.

3.3 Definición del entorno

El entorno representa una cartera simulada que evoluciona a partir de datos históricos. Cada episodio contiene una secuencia de observaciones de mercado y un estado interno formado por saldo disponible, posiciones abiertas, posiciones bloqueadas, fechas de desbloqueo y métricas de exposición.

Algoritmo 1 Ciclo general de toma de decisiones

- 1: Recoger observaciones recientes del mercado.
 - 2: Actualizar el estado de cartera y las variables de riesgo.
 - 3: Generar una observación para cada agente.
 - 4: Obtener acciones de Scout, Trader y Portfolio.
 - 5: Agregar acciones y aplicar restricciones de riesgo.
 - 6: Simular la decisión final sobre la cartera.
 - 7: Calcular recompensa, métricas y trazabilidad del episodio.
-

3.3.1 Estado y observaciones

Las observaciones combinan variables de mercado y variables de cartera. Entre las variables de mercado se incluyen precios de Steam y BUFF163 normalizados a EUR, precios originales en su moneda nativa, volumen, liquidez, spread bruto, spread neto, buy orders, edad de la variante y señales temporales como retornos, medias móviles, RSI o desviaciones de ventas. Entre las variables de cartera se incluyen efectivo disponible, capital bloqueado, posiciones abiertas, exposición por activo y estado de desbloqueo.

La salida del modelo supervisado no se interpreta como orden de compra. Se incorpora como una característica adicional de observación: una probabilidad calibrada de que una oportunidad mantenga un spread rentable en el horizonte definido. De esta forma, los agentes pueden utilizar la señal supervisada sin quedar reducidos a copiarla.

3.3.2 Espacio de acciones

El espacio de acciones se mantiene deliberadamente simple. Las acciones principales son comprar, vender, mantener y observar. En la parte de compra o venta puede añadirse un tamaño de posición discreto para limitar el número de combinaciones. Esta simplificación permite entrenar y depurar el entorno antes de introducir acciones continuas o reglas de asignación más complejas.

3.3.3 Transición del entorno

Cuando se ejecuta una compra simulada, el saldo disponible disminuye y se abre una posición. Esa posición queda bloqueada durante el periodo de *trade hold*. Mientras está bloqueada no puede venderse, pero sí afecta a la exposición y al capital inmovilizado. Cuando se ejecuta una venta simulada, el simulador calcula el ingreso neto aplicando comisiones, conversión de moneda y reglas de valoración. El estado de cartera se actualiza después de cada paso.

3.4 Agentes del sistema

La arquitectura distingue entre agentes extractores y agentes MARL. Los extractores no se entrenan mediante aprendizaje por refuerzo y no deciden operaciones. Su trabajo es adquirir datos desde Steam, BUFF163, SteamDT, OCR o CSV/API y transformarlos en observaciones persistidas. Los agentes MARL, en cambio, operan dentro del entorno de simulación.

3.4.1 Scout

Scout se encarga de detectar oportunidades. Su observación está orientada a señales de precio, spread, liquidez y probabilidad supervisada. Su papel no es decidir toda la operación, sino marcar qué activos parecen merecer atención dentro del episodio.

3.4.2 Trader

Trader decide la acción operativa principal: comprar, vender, mantener u observar. Para ello considera la señal de Scout, el estado de mercado, el posible beneficio neto y el tamaño de posición permitido. Su responsabilidad está más cerca de la ejecución de una oportunidad concreta.

3.4.3 Portfolio

Portfolio controla la coherencia global de la cartera. Su observación incorpora saldo, capital bloqueado, posiciones abiertas, exposición máxima y restricciones de riesgo. Este agente puede penalizar o bloquear decisiones que parezcan rentables de forma aislada pero que deterioren el perfil global de la cartera.

3.5 Función de recompensa

La recompensa debe reflejar el objetivo real del sistema: mejorar el valor neto de la cartera sin ignorar riesgo ni fricción. Una recompensa basada solo en comprar barato o en detectar subida de precio sería incompleta, porque no tendría en cuenta si la operación puede cerrarse, si el capital queda bloqueado o si el spread desaparece antes de poder vender.

3.5.1 Recompensa individual

Las señales individuales pueden ayudar a entrenar comportamientos especializados. Scout puede recibir recompensa por identificar oportunidades que más adelante resultan útiles; Trader por ejecutar acciones con beneficio neto; y Portfolio por evitar sobreexposición, operaciones ilíquidas o capital bloqueado excesivo. Estas recompensas no sustituyen al objetivo global, pero pueden guiar el aprendizaje de cada rol.

3.5.2 Recompensa cooperativa

La recompensa cooperativa se vincula al valor total de la cartera simulada. Debe considerar beneficio realizado, valoración de posiciones abiertas, capital bloqueado, drawdown, operaciones rechazadas por riesgo y costes de transacción. En la implementación actual, el entorno MARL ya expone métricas como recompensa media, operaciones, efectivo, beneficio realizado, drawdown, capital bloqueado y posiciones abiertas.

3.6 Restricciones de riesgo

Las restricciones de riesgo se introducen antes de escalar el entrenamiento. Las más importantes son: no usar capital inexistente, limitar exposición por activo o plataforma,

rechazar operaciones con liquidez insuficiente, respetar el *trade hold*, controlar el capital bloqueado y separar claramente simulación de operación real. Estas reglas son sencillas, pero evitan que los agentes aprendan políticas imposibles de ejecutar en el mercado real.

Configuración del entorno y datos

4.1 Fuentes de datos

El sistema trabaja con varias fuentes porque el mercado de CS2 está fragmentado. Ninguna fuente aislada contiene toda la información necesaria para decidir. Steam aporta precio, histórico y señales del mercado principal; BUFF163 permite comparar oportunidades y buy orders; SteamDT ayuda a descubrir candidatos; el OCR permite capturar datos cuando no existe una API cómoda; y los CSV o importaciones manuales sirven para integrar muestras controladas o material histórico.

4.1.1 Mercado de Steam

Steam actúa como una de las fuentes principales de precio e histórico. En el proyecto se capturan variantes reales de artículos, precios actuales, moneda, buy orders y puntos históricos cuando están disponibles. La información se normaliza para que pueda combinarse con otras plataformas y para evitar que las decisiones dependan de nombres o formatos propios de una fuente.

4.1.2 Mercados externos

BUFF163 se utiliza como mercado externo de comparación. Sus precios se reciben principalmente en CNY, por lo que el sistema conserva tanto el valor original como su conversión a EUR. Esta doble representación permite auditar el dato original y, al mismo tiempo, entrenar modelos sobre una moneda canónica. SteamDT se utiliza como apoyo para descubrir candidatos y estrategias de búsqueda, no como fuente única de decisión.

4.2 Adquisición y persistencia

4.2.1 Pipeline de adquisición

La adquisición se organiza como una capa de agentes extractores. Estos procesos pueden ejecutarse como comandos CLI, jobs o servicios programados. Su función es capturar datos, normalizarlos, validar errores y guardarlos con trazabilidad. El flujo incluye scraping, refresco de históricos, OCR, importación manual y un bucle operativo local que puede ejecutar scraping y actualización periódica.

La adquisición periódica contempla cookies o sesiones, delays configurables, límites de concurrencia, reintentos, deduplicación y registro de anomalías. Esta parte es importante porque las fuentes no siempre son estables: pueden cambiar el formato, limitar el ritmo de acceso o devolver información incompleta.

4.2.2 Modelo de persistencia

La persistencia se apoya en Supabase/Postgres. La decisión de diseño es tratar la base de datos como fuente de verdad y mantener el histórico de forma append-only siempre que sea posible. Las tablas operativas principales son `market_items` y `market_history_points`. La primera mantiene una fila por variante real de artículo y estado actual por plataforma; la segunda guarda series temporales en formato largo por artículo, plataforma, fecha y métrica.

También se define un esquema canónico más amplio con entidades como `assets`, `platforms`, `market_observations`, `predictions`, `investment_decisions` y `simulated_positions`. Esta separación permite evolucionar desde el esquema operativo actual hacia una representación más trazable y auditable.

4.3 Preparación de características

4.3.1 Variables de mercado

Las características de mercado incluyen precios Steam y BUFF normalizados, precios originales, spread bruto, spread neto, buy orders, volumen, liquidez, número de listings, antigüedad de variante y señales temporales. En la fase supervisada se exploraron variables de momentum y reversión como retornos a 1, 3 y 7 días, distancia a medias móviles, RSI, volatilidad y desviaciones de ventas.

La exploración inicial detectó que algunas señales temporales aportan información moderada, mientras que ciertas variables categóricas de alta cardinalidad, como identificadores concretos de skins, deben tratarse con cuidado para evitar sobreajuste. Por ello, el entrenamiento separa variables de evaluación y variables de entrenamiento, y permite realizar ablations sin reconstruir todo el dataset.

4.3.2 Variables de cartera

Las variables de cartera proceden del simulador. Incluyen saldo disponible, valor de posiciones abiertas, capital bloqueado por *trade hold*, fecha de desbloqueo, exposición por activo, exposición por plataforma, beneficio realizado y drawdown. Estas variables son necesarias para que el agente Portfolio no decida únicamente por oportunidad local, sino por estado global de la cartera.

4.4 Simulador de cartera

El simulador reproduce las reglas económicas que antes estaban dispersas en el Excel operativo. No intenta copiar la hoja de cálculo, sino migrar sus reglas útiles a código comprobable: conversión EUR/CNY, comisiones, beneficio neto, rentabilidad porcentual, fecha de desbloqueo y capital bloqueado.

4.4.1 Operaciones permitidas

Las operaciones básicas son comprar, vender y mantener. Una compra abre una posición con precio de entrada, cantidad, plataforma y fecha de bloqueo. Una venta

cierra una posición si está disponible y calcula el beneficio realizado. Mantener no modifica la posición, pero puede afectar al rendimiento si el mercado se mueve o si el capital sigue inmovilizado.

4.4.2 Comisiones y valoración

El Excel operativo utiliza reglas de referencia que se han convertido en hipótesis de simulación. BUFF se modela con un factor neto de venta de 0,975. Steam usa una comisión de mercado del 0,87 y, en escenarios conservadores de salida a banco, puede aplicarse una pérdida adicional del 20%. La conversión inicial simplificada usa 1 EUR = 8 CNY, aunque queda pendiente versionar tipos de cambio por fecha si se incorporan fuentes externas.

El *trade hold* se modela de forma conservadora como ocho días desde la compra, equivalente a siete días más una ventana de desbloqueo. Esta simplificación evita declarar vendible una posición demasiado pronto.

4.5 Interfaz de consulta

El proyecto incluye una interfaz web como apoyo operativo. Su objetivo no es vender el sistema como producto final, sino hacer visible el estado de adquisición, modelos, simulador, restricciones y recomendaciones. La interfaz debe permitir responder rápidamente qué datos existen, qué modelo está activo, qué señales se están generando y por qué una oportunidad se recomienda, se observa o se descarta.

Experimentación y análisis

5.1 Diseño experimental

La experimentación se ha desarrollado por fases. Primero se migraron reglas económicas y estructuras de datos. Después se construyeron datasets supervisados para estudiar señales de mercado. Más adelante se creó un dataset de trading basado en precios Steam/BUFF y se implementó el entorno multiagente. Esta secuencia evita entrenar agentes MARL sobre un entorno que todavía no representa bien el problema económico.

5.1.1 Escenarios de evaluación

Se han usado dos tipos de datasets. El primero predice dirección de mercado sobre histórico disponible y permite estudiar señales temporales de forma amplia. El segundo intenta modelar oportunidades reales de trading Steam/BUFF con precios normalizados, fees y horizonte temporal. Este segundo dataset es más cercano al objetivo del TFM, pero también es más exigente porque necesita histórico alineado entre plataformas.

5.1.2 Estrategias de comparación

La comparación experimental se plantea en tres niveles. El primero es un baseline simple o dummy para comprobar que los modelos aprenden algo más que la tasa base. El segundo compara modelos supervisados tabulares como regresión logística, random forest e histogram gradient boosting. El tercero, todavía pendiente de evaluación completa, comparará la arquitectura MARL frente a baselines de agente único y ablations con y sin probabilidad supervisada.

5.2 Métricas

5.2.1 Métricas de rentabilidad

Las métricas de rentabilidad principales son beneficio neto, retorno porcentual, saldo final y número de señales operativas. En los modelos supervisados se usan además precisión, recall, F1, ROC-AUC, PR-AUC y Brier score. Para el uso operativo se da especial importancia a la precisión en umbrales altos, porque una señal menos frecuente pero más fiable puede ser más útil que muchas señales ruidosas.

5.2.2 Métricas de riesgo

Las métricas de riesgo incluyen drawdown, capital bloqueado medio, posiciones abiertas, exposición por activo o plataforma, operaciones rechazadas por riesgo e impacto del *trade hold*. Estas métricas son necesarias para evaluar si una estrategia es realmente utilizable y no solo rentable en una métrica aislada.

5.3 Resultados

5.3.1 Resultados del modelo supervisado

La fase supervisada produjo varios experimentos preliminares. En el dataset reciente de un año, los modelos random forest obtuvieron señales útiles en umbrales altos, aunque con un número limitado de casos. La selección por precisión a umbral 0,8 resultó más alineada con el objetivo operativo que seleccionar solo por ROC-AUC.

Experimento	Modelo	Umbral	Resultado en test
Smoke reciente	Random forest d18	0,8	Precisión 0,946 con 92 señales
Selección por precisión	Random forest d10	0,8	Precisión 0,961 con 77 señales
Default operativo	Random forest d18	0,85	Precisión 0,962 con 53 señales

Tabla 5.1: Resultados supervisados preliminares sobre el dataset reciente.

Estos resultados no se consideran definitivos. El número de señales en umbrales altos es reducido y no permite afirmar que el sistema generalice por artículo o por periodo. Sí indican que el pipeline de entrenamiento, calibración, selección por umbral y evaluación temporal funciona.

5.3.2 Dataset de trading real

El dataset `trading_profit_v1` se generó desde `market_history_points` usando precios Steam/BUFF, comisiones y horizonte temporal. La primera versión produjo 2.331 ejemplos y 50 artículos, pero con desbalance extremo en validación y test: solo un caso positivo en cada split para la dirección analizada. Por este motivo, las métricas de ese entrenamiento no se usan como evidencia de rendimiento operativo.

El análisis posterior mostró que el histórico de BUFF empezó a estar mejor alineado tras corregir el parser y permitir backfill de puntos históricos. Aun así, la dirección más natural de arbitraje, comprar en BUFF y vender en Steam, todavía no generaba ejemplos suficientes a ocho días porque faltaba histórico futuro Steam alineado con los puntos BUFF recientes.

5.3.3 Resultados de la arquitectura multiagente

El entorno multiagente ya existe como smoke funcional. Se implementó un entorno PettingZoo con tres políticas, un wrapper para RLlib y un comando de entrenamiento

corto con PPO multiagente. El smoke ejecuta entrenamiento, guarda checkpoint y devuelve métricas operativas como recompensa media, número de operaciones, efectivo, beneficio realizado, drawdown, capital bloqueado y posiciones abiertas.

Sin embargo, el entrenamiento actual no debe presentarse como validación final de MAPPO con CTDE completo. El estado central está definido y expuesto, pero queda pendiente conectar el crítico centralizado de forma completa y comparar contra baselines. Esta distinción es importante para no confundir un flujo ejecutable con una evidencia experimental cerrada.

5.4 Discusión

5.4.1 Casos favorables

El trabajo realizado muestra que el sistema ya puede integrar fuentes, construir datasets, entrenar modelos supervisados, calibrar probabilidades, simular reglas económicas y ejecutar un smoke multiagente. La separación entre adquisición, predicción, simulación y decisión ha resultado útil porque permite avanzar por partes y detectar bloqueos concretos sin rehacer toda la arquitectura.

5.4.2 Limitaciones observadas

La principal limitación actual es la disponibilidad de histórico alineado entre Steam y BUFF. Sin suficientes ejemplos positivos en validación y test, cualquier métrica de trading puede ser engañosa. También existe riesgo de sobreajuste en variables categóricas de alta cardinalidad y en señales de precisión alta con pocos casos. Por último, el entorno MARL necesita una validación experimental más amplia antes de poder concluir si la arquitectura multiagente mejora realmente a una estrategia simple.

Conclusiones

6.1 Conclusiones principales

El desarrollo realizado hasta el momento permite concluir que el problema no puede reducirse a una predicción aislada de subida o bajada de precio. En mercados como el de *Counter-Strike 2*, la decisión depende de fricciones muy concretas: comisiones, conversión de moneda, liquidez, *trade hold*, capital bloqueado y riesgo de cartera. Por ello, la arquitectura propuesta necesita combinar adquisición de datos, simulación económica y decisión multiagente.

También se ha comprobado que la separación por capas facilita el avance del proyecto. Los agentes extractores se encargan de observar el mercado; el modelo supervisado produce señales probabilísticas; el simulador aplica reglas económicas; y los agentes MARL operan dentro de un entorno controlado. Esta división reduce el acoplamiento y permite validar cada parte por separado.

6.2 Cumplimiento de objetivos

Se ha avanzado en la mayoría de objetivos técnicos: existe un monorepo modular, un modelo de datos operativo, pipelines de adquisición, análisis del Excel original, construcción de datasets, entrenamiento supervisado, simulador de cartera y un entorno MARL funcional para smoke tests. También se han definido agentes Scout, Trader y Portfolio, junto con restricciones de riesgo y métricas de evaluación.

Queda pendiente completar la validación experimental fuerte. En particular, todavía falta entrenar y comparar la arquitectura MARL frente a baselines de agente único, ejecutar ablations con y sin probabilidad supervisada y evaluar con datos no vistos suficientemente ricos.

6.3 Limitaciones

La limitación más importante es la escasez de histórico alineado entre plataformas para algunas direcciones de trading. El dataset real más cercano al objetivo operativo presenta pocos positivos en validación y test, por lo que no permite defender todavía una mejora robusta. Además, algunos resultados supervisados tienen alta precisión en umbrales altos, pero con pocas señales, lo que obliga a interpretarlos con prudencia.

Otra limitación es que el smoke multiagente demuestra que el flujo ejecuta, pero no que el sistema haya aprendido una política superior. Para llegar a esa conclusión se necesita una evaluación reproducible con más episodios, más datos, baselines justos y métricas de riesgo.

6.4 Trabajo futuro

Las líneas inmediatas de trabajo son acumular más histórico Steam/BUFF alineado, regenerar el dataset de trading con suficientes positivos, validar el modelo supervisado por ventanas temporales y entrenar MARL con una configuración CTDE completa. Después será necesario comparar contra baselines, estudiar ablations, generar gráficas de resultados y cerrar una discusión experimental honesta sobre cuándo la arquitectura multiagente aporta valor y cuándo no.

Bibliografía

- [1] Eric Liang, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph E. Gonzalez, Michael I. Jordan, and Ion Stoica. Rllib: Abstractions for distributed reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning*, 2018.
- [2] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. <https://arxiv.org/abs/1707.06347>, 2017.
- [3] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- [4] Justin K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis Santos, Rodrigo Perez, Caroline Horsch, Clemens Diefendahl, Niall L. Williams, Yashas Lokesh, Praveen Ravi Hines, and Niall Ravi. Pettingzoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, 2021.

Anexos

A.1 Estructura del repositorio

El repositorio se organiza como un monorepo. La carpeta **apps** contiene aplicaciones y comandos ejecutables. La carpeta **packages** contiene la lógica reutilizable. La carpeta **docs** recoge decisiones y documentación técnica. La carpeta **tests** contiene pruebas unitarias e integración. La carpeta **backlog** mantiene el estado de tareas realizadas, pendientes y en progreso.

Los módulos principales son:

- **apps/acquisition**: extractores, scraping, OCR, importadores y SteamDT.
- **apps/cli**: comandos de construcción de datasets, entrenamiento, scraping y evaluación.
- **packages/contracts**: contratos Pydantic compartidos.
- **packages/persistence**: conexión, repositorios y persistencia.
- **packages/prediction**: entrenamiento e inferencia supervisada.
- **packages/simulation**: reglas económicas, cartera, riesgo y backtesting.
- **packages/marl**: entorno multiagente, wrappers PettingZoo/RLlib y entrenamiento.
- **packages/vision**: OCR, preprocesado y parsing visual.

A.2 Configuración del entorno

El entorno se define en `pyproject.toml`. El proyecto usa Python, Supabase/Postgres, Pytest, Ruff, Mypy y dependencias opcionales para MARL como Ray/RLlib, PettingZoo, Gymnasium y PyTorch. Para desarrollo local existe un `docker-compose.yml` con Postgres.

Comandos representativos:

```
python -m pytest tests/unit
python -m ruff check
python -m mypy apps packages tests/unit
python scrape_flow.py --show-browser --persist
python -m apps.cli.auto_scrape_loop --once
```

A.3 Parámetros de entrenamiento

Los experimentos supervisados usan splits temporales, calibración de probabilidades y selección por precisión operativa en umbrales altos. En los smoke tests se compararon modelos dummy, regresión logística, random forest e histogram gradient boosting. El entrenamiento MARL usa RLlib con un entorno multiagente y tres políticas: Scout, Trader y Portfolio.

A.4 Modelo de datos

El modelo operativo actual gira alrededor de dos tablas principales: `market_items` y `market_history_points`. La primera representa variantes reales de artículos y su estado actual por plataforma. La segunda almacena series temporales en formato largo, con una fila por artículo, plataforma, fecha y métrica. Esta estructura evita mezclar métricas distintas y permite añadir nuevas señales sin alterar la forma general del histórico.

A.5 Pruebas implementadas

El proyecto incluye pruebas para parsing de mercado, OCR, persistencia, datasets supervisados, entrenamiento supervisado, simulador de cartera, riesgo, entorno MARL, wrappers RLlib y flujos de scraping. La suite se ejecuta de forma recurrente con Pytest, Ruff y Mypy.

A.6 Resultados adicionales

Los resultados intermedios se guardan en `model-runs` y los datasets derivados en `data/datasets`. Estas carpetas permiten conservar reportes de cobertura, exploración de características, métricas de entrenamiento, umbrales seleccionados y salidas de smoke tests sin mezclar los artefactos pesados con el código principal.